carried out as follows:

```
using Flux
f(x) = 3x^2 + 2x + 1;
df(x) = gradient(f, x)[1]; # df/dx = 6x + 2
df(4)
```

When a function has many parameters, the gradient for each can be computed as follows:

```
f(x, y) = sum((x .- y).^2);
x = [2, 1];
y = [2, 0]
gs = gradient(params(x, y)) do
         f(x, y)
     end
gs[x]
```

The output is a vector, as shown below:

```
gs[x]
2-element Vector{Int64}:
 0
 2
```

Adding a regularisation term

In machine learning, regularisation is a technique to handle overfitting. Overfitting is a problem when your model performs well with the training data but fails when exposed to novel data. The problem here is the lack of generalisation. There are various regularisation techniques. Flux enables adding regularisation terms very simply. A code snippet is as follows:

```
m = Chain(
  Dense(28^2, 128, relu),
  Dense(128, 32, relu),
  Dense(32, 10))

sqnorm(x) = sum(abs2, x)

loss(x, y) = logitcrossentropy(m(x), y) + sum(sqnorm, Flux.
params(m))

loss(rand(28^2), rand(10))
```

In the aforementioned example, L2 regularisation is applied by taking the squared norm of the parameters.

GPU support

Flux has good support for GPUs. This can be achieved using CUDA. The CUDA.CuArray shall be used for this purpose. A sample code segment is as shown below:

```
using CUDA
W = cu(rand(2, 5)) # a 2×5 CuArray
b = cu(rand(2))
predict(x) = W*x .+ b
loss(x, y) = sum((predict(x) .- y).^2)
x, y = cu(rand(5)), cu(rand(2)) # Dummy data
loss(x, y)
```

GeometricFlux

As stated earlier, GeometricFlux is a geometric deep learning library for Flux. The handling of geometric deep learning using this library can be done similar to handling the traditional models, as shown below:

```
model = Chain(GCNConv(fg, 1024 => 512, relu),
              Dropout(0.5),
              GCNConv(fg, 512 => 128),
              Dense(128, 10))
## Loss
loss(x, y) = logitcrossentropy(model(x), y)
accuracy(x, y) = mean(onecold(model(x)) .== onecold(y))


## Training
ps = Flux.params(model)
train_data = [(train_X, train_y)]
opt = ADAM(0.01)
evalcb() = @show(accuracy(train_X, train_y))

Flux.train!(loss, ps, train_data, opt, cb=throttle(evalcb,
10))
```

This article has provided a simple introduction to Flux, which is fully built in Julia. The support for automatic differentiation and GPU will help you to explore Flux for your projects. END

References

[1] *https://fluxml.ai/*
[2] *https://fluxml.ai/Flux.jl/stable/*

By: Dr K.S. Kuppusamy

The author is an assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University, Pondicherry. He has 14+ years of teaching and research experience in academia and industry. His research interests include accessible computing, Web information retrieval, and mobile computing.